Google

Linux Kernel Tracing

Technical Update: Recent Developments



Linux Plumbers Conference 2025

Dec, 13, 2025 Tokyo

Masami Hiramatsu (Google) <mhiramat@kernel.org>

Agenda

- Foundational Stability & Access
- Tracer Updates
- Probes/Events Updates
- Conclusion

Foundational Stability & Access

Enhancing resilience and broadening access for production tracing environments.

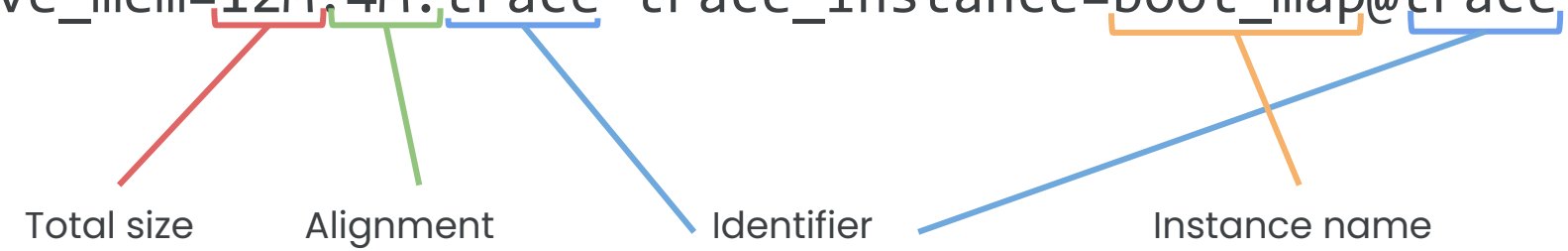
Persistent Ring Buffer (Post-Crash Analysis)

- **Crash Resilience:** The buffer can be configured to persist across reboots, allowing for post-mortem analysis of crashes where the kernel cannot save log data to disk.
- **Reserved Memory:** Requires reserving a dedicated memory region (via `reserve_mem` kernel parameter) to ensure the trace data is kept during a crash/reboot sequence (depends on the bootloader).
- **KASLR & Module Data:** Recent updates enable the saving and reconstruction of critical context data like the Kernel Address Space Layout Randomization (KASLR) offset and module load addresses, making symbol resolution possible in the next boot.
- **Use Case:** Non-destructive tracing for intermittent, hard-to-reproduce crashes, providing the final sequence of kernel events.

Persistent Ring Buffer (Setup)

Usage: Adding kernel cmdline:

“reserve_mem=12M:4M:trace trace_instance=boot_map@trace”



“Make a persistent ring buffer instance as “boot_map” which size is 12 MiB and it is 4 MiB aligned.”

Persistent Ring Buffer (Use case)

Example: debug hw_breakpoint hung up on arm64[1]

Setup a custom overflow handler to hw_breakpoint in the kernel, it freezes machine.

```
# echo "w:jiffies_update w@jiffies" >> dynamic_events
# echo 1 > events/wprobes/jiffies_update/enable
(frozen)
```

Google

Persistent Ring Buffer (Use case)

Example: debug hw_breakpoint hung up on arm64[1]

Setup a custom overflow handler to hw_breakpoint in the kernel, it freezes machine.
-> Add **trace_printk**() and record it on the persistent ring buffer

```
# echo 1 > instances/boot_map/events/enable
# echo "w:jiffies_update w@jiffies" >> dynamic_events
# echo 1 > events/wprobes/jiffies_update/enable
(frozen -> manually reset)
# less instances/boot_map/trace
```

-> Same events repeated with trace_printk() on the same address (no other events)

Google

Backup of Persistent Ring Buffer [6.19]

Usage: Adding kernel cmdline:

```
“reserve_mem=12M:4M:trace trace_instance=boot_map@trace,sched:*
trace_instance=backup=boot_map”
```



This copies the “boot_map” persistent instance into “backup” instance.
Thus, user can continue to use (overwrite) the persistent instance soon after boot.
e.g. schedule events are recorded on the persistent instance always.

Tracer Updates

Recording more useful trace data

Google

Syscall Tracer with User Context Access

Access User Data in Syscalls

Syscall Events are no longer limited to kernel data and register states. **BTF (BPF Type Format)** allows tracers to read full user-space arguments, including strings, at the point of a syscall entry or exit. **[Steven Rostedt]**

```
              sh-107     [002] .....    77.339118: sys_geteuid -> 0x0
              sh-107     [002] .....    77.339122: sys_write(fd: 2, buf: 0x8ad5960
(2f:73:79:73:2f:6b:6
5:72:6e:65:6c:2f:74:72:61:63:69:6e:67:20:23:20) "/sys/kernel/tracing # ", count:
0x16)
              sh-107     [002] .....    77.340181: sys_write -> 0x16
```

Use Case: Comprehensive lightweight **strace** (e.g., logging network or file data passed to `read`/`write` syscalls)

Function Graph Tracer with Parameters (Fgraph + BTF)

Function graph tracer can access function parameters and return values **[Steven Rostedt]**

```
# echo function_graph > current_tracer
# echo 1 > options/funcgraph-args
# echo 1 > options/funcgraph-retva
# cat trace
...
fetch_next_timer_interrupt(basej=0x100479251,
basem=0x489f4514f00, base_local=
0xffff88807d8a7900, base_global=0xffff88807d8a8bc0,
tevt=0xffffc900000c3e20); /* ret=0x100479440 */
```

- **Context-Aware Flow:** The Func-graph tracer, which maps function call hierarchy, now integrates with BTF to extract and display parameter values at function entry.
- **Simplified Debugging:** Transforms raw stack traces into high-level, human-readable call graphs, dramatically simplifying the process of debugging complex control flow paths.
- **Example Output:** Instead of registers, the trace displays:

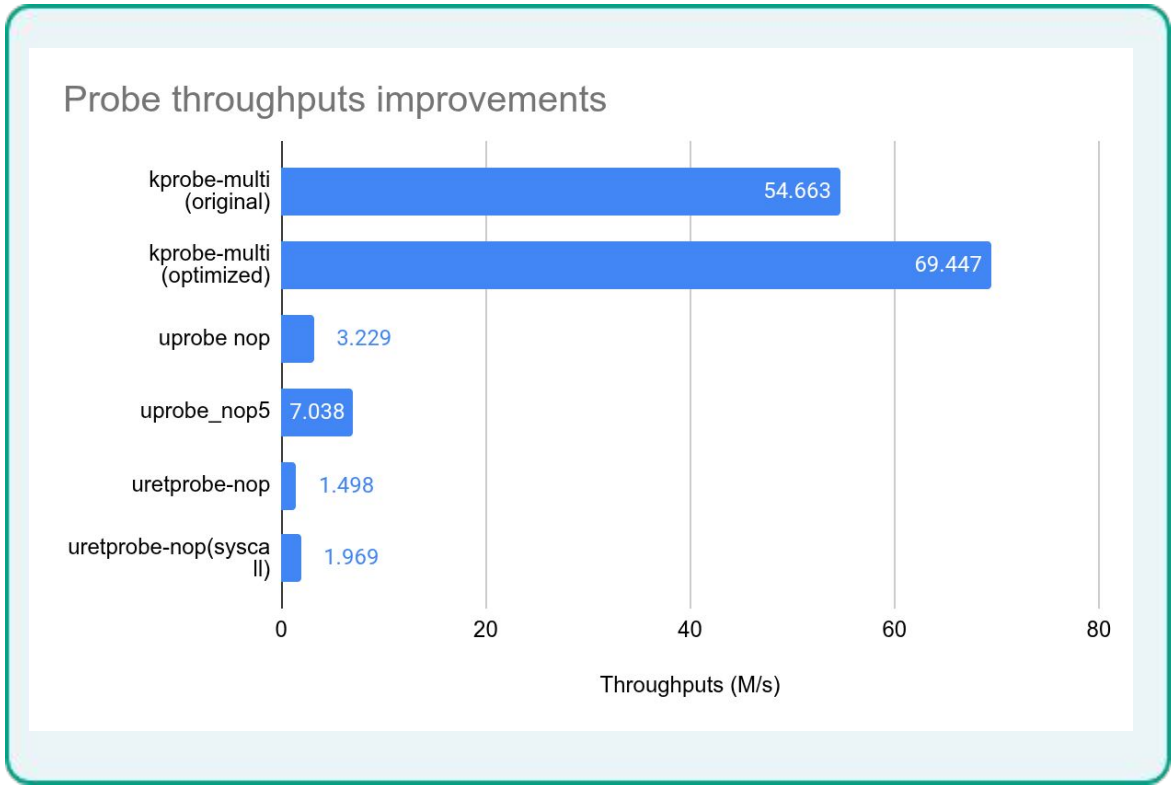
Probes/Events Updates

Achieving trapless performance and unifying function instrumentation logic.

Google

Uprobe/Fprobe Performance Optimization

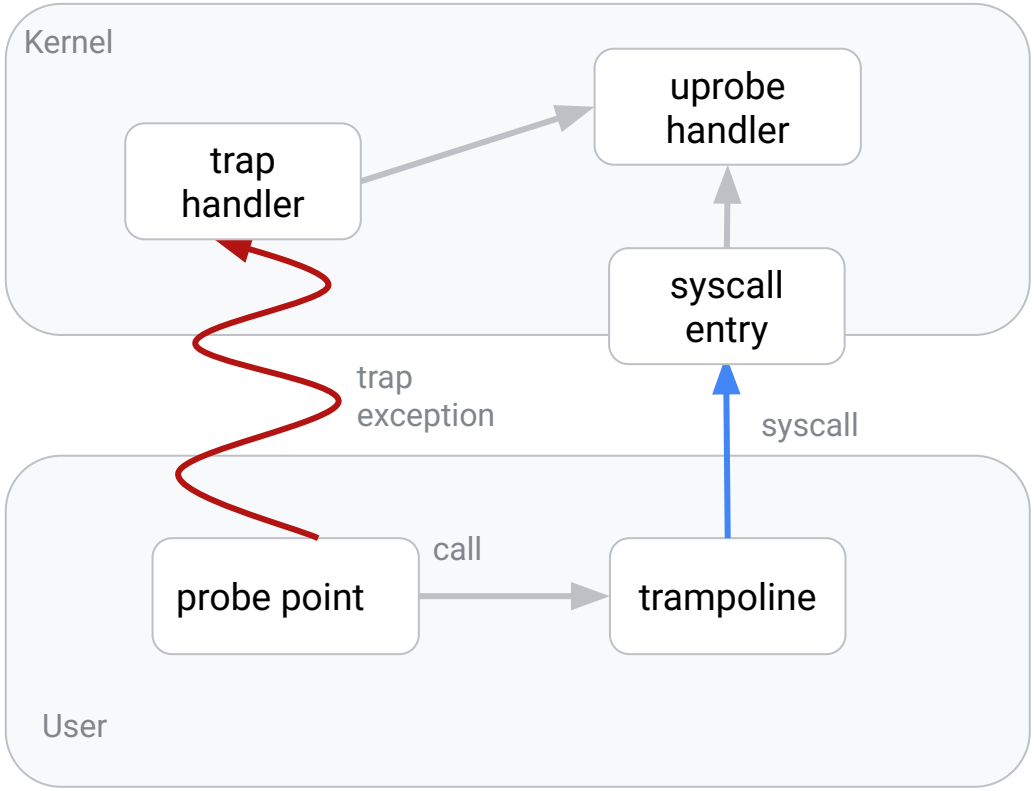
- **Fprobe Refactoring:** Fprobe is being implemented with ftrace(for entry-only use case) and "*rhashtbl*"(for multi-function use case). **[Menglong Dong]**
- **Trapless U(ret)probes:** Efforts to maximize the use of optimized, trapless uprobes (using syscall trampoline) to avoid expensive CPU trap exceptions. **[Jiri Olsa]**
- **Fprobe wildcard events:** A single fprobe event is allowed to specify multiple functions to probe by wildcard and function-list(Experimental). **[Seokwoo Chung (Ryan)]**



U(ret)probe Syscall Optimization

Introduces `uprobe` and `uretprobe` syscall

- These are used transparently from uprobe probe point.
- Probe point generates a trampoline and call that trampoline code. (for all uretprobe, but for uprobes, it is limited on 5-byte nop.)
  - trampoline code uses a syscall to call kernel instead of a trap.



Google

USDT vs Trace_marker (Old)

USDT (User-space Static Defined Tracepoint): Marking tracepoint with arguments by NOP. Actual trace is done by external probes (e.g. uprobe)

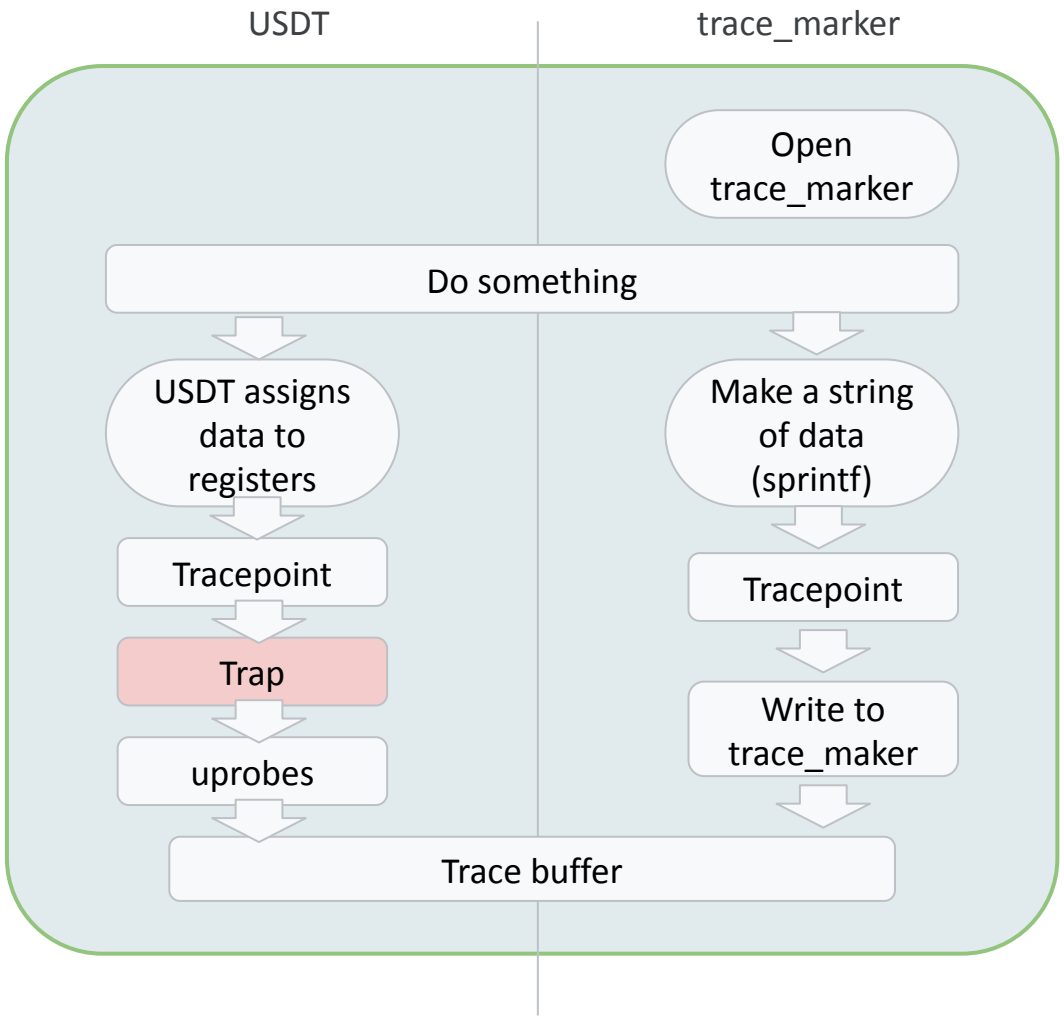
Pros: No preparation for open, record raw data

Cons: higher overhead by trap.

Trace marker: Write an event string/data into kernel via trace_marker file.

Pros: Low overhead

Cons: Need write permission for trace_marker, ftrace can not filter by data (because it is string), all events are the same type.



Google

USDT vs Trace_marker (New)

USDT (User-space Static Defined Tracepoint): Marking tracepoint with arguments by NOP. Actual trace is done by external probes (e.g. uprobe)

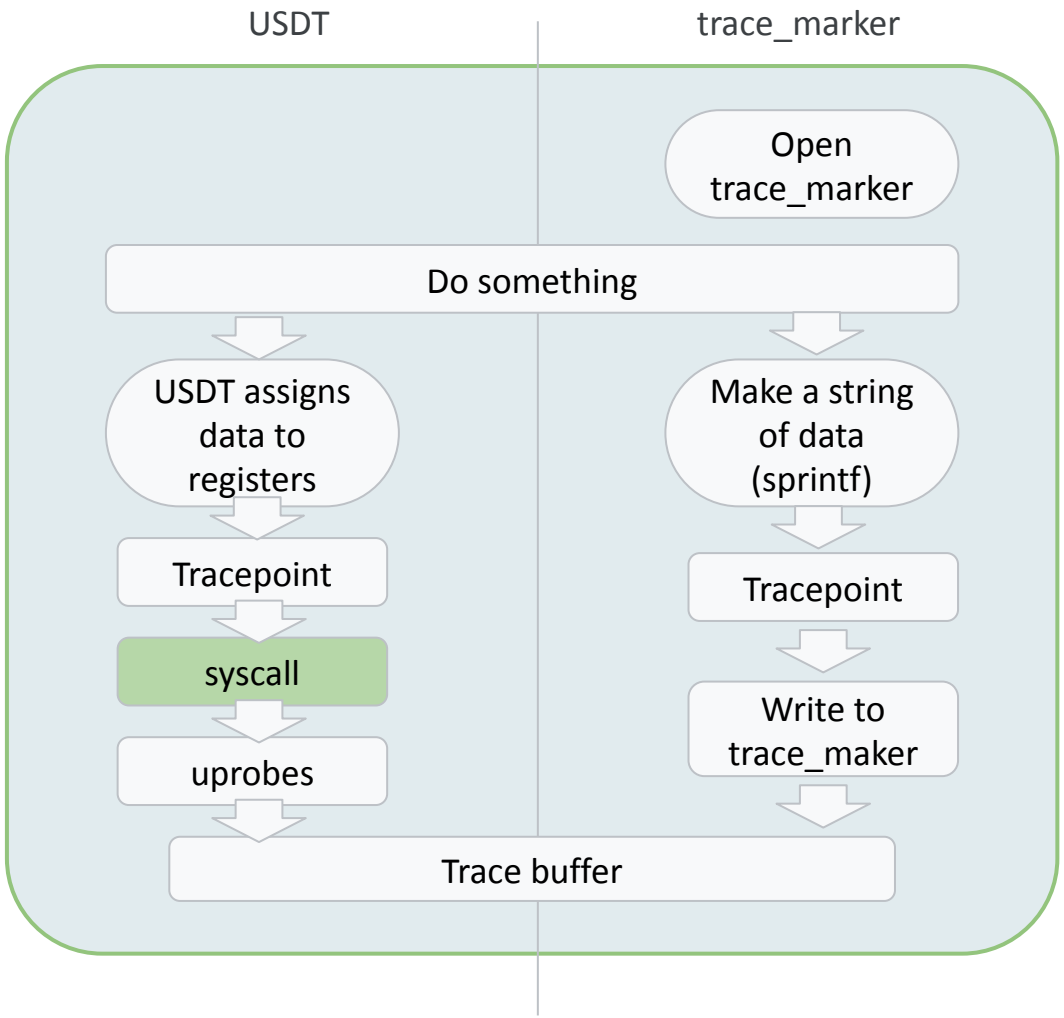
Trapless Uprobes: Replace a 5-byte NOP with a call to trampoline, which calls single system-call to probe registers.

Pros: No preparation for open, record raw data,

Low overhead

~~**Cons:** higher overhead by trap.~~

Benefit: **Tracing without preparation/permission, ftrace full feature**



Google

Override Events: Tprobes & Eprobes

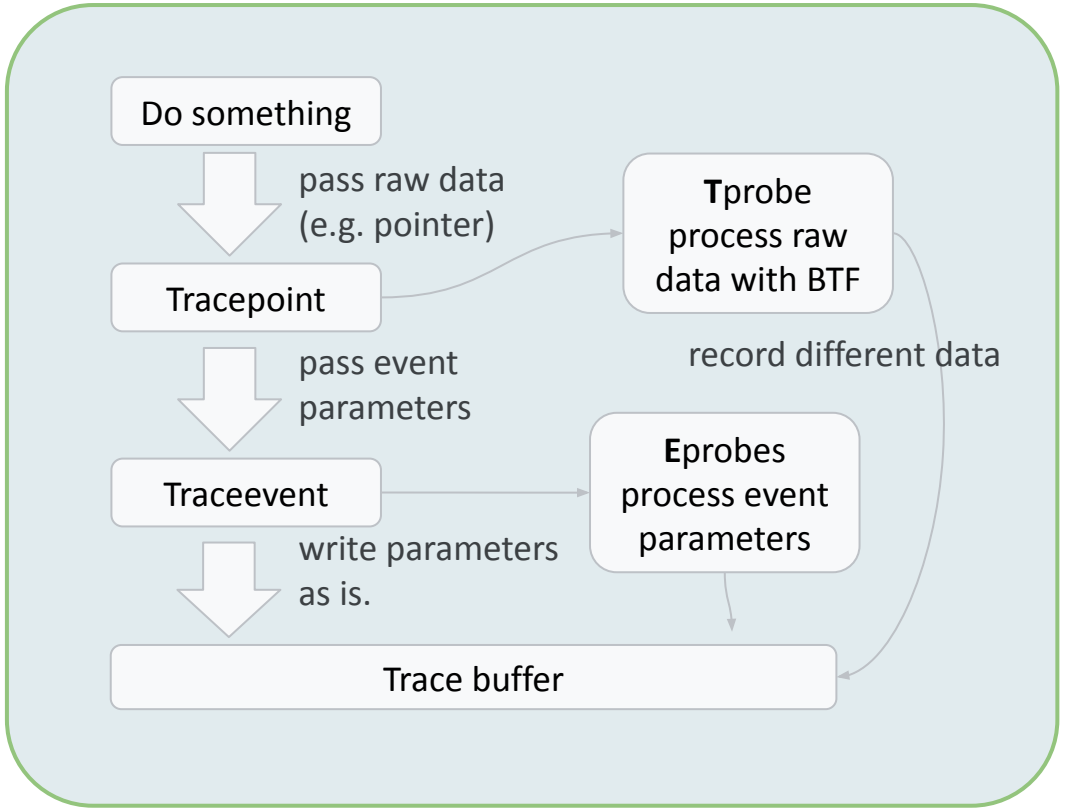
Tprobes and Eprobes are able to override other events, but the layer is different.

Tprobes (Tracepoint Probes)

Attach to kernel Tracepoints. Primary use is for **BTF-aware dereferencing** of pointers/fields, allowing structured data extraction from static event points.

Eprobes (Event Probes)

A probe that hooks into an **existing kernel trace event**. Its key power is the ability to securely **dereference and extract data** pointed to by fields within that trace event's structure.



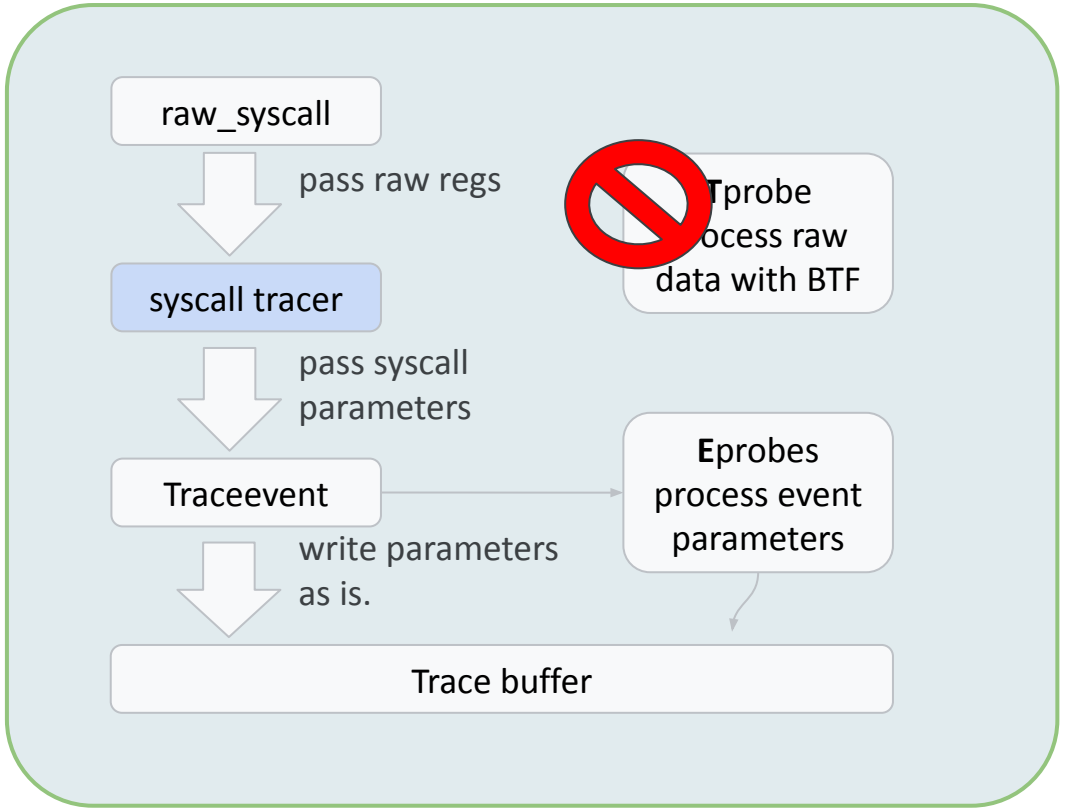
Google

Override Events: Why Tprobes & Eprobes?

Syscall events are different mechanism

Tprobe is **generally usable** for most events. But only syscalls (not raw_syscall) events can not hook. Syscall event is generated directly from **syscall tracer.**

Eprobe is still available for syscall events, because traceevents are there!



Google

Wprobe: Hardware Watchpoints (Experimental)

Wprobe Functionality (Hardware Layer)

- **Mechanism:** Wprobe utilizes the CPU's **hardware debug/watchpoint registers** (a.k.a. hw_breakpoint) to monitor a memory address.
- **Trigger Types:** Can be configured to fire on memory **access (read)** and/or **write (store)** operations.
- **Static Specification:** The probe can be attached to the memory address of a global or static kernel symbol, providing tracing capability without altering the code.
- **Benefit:** **Tracing specific memory access**, as monitoring is handled directly by the hardware debug unit.

```
# echo "w:jiffies_access rw@jiffies" >> dynamic_events
# echo 1 > events/wprobes/jiffies_access/enable
# tail trace
            tail-119     [003] ..Zff   252.603804:
jiffies_access: (tick_nohz_handler+0x73/0x110)
            tail-119     [003] ..Zff   252.603808:
jiffies_access: (update_process_times+0x32/0xd0)
            tail-119     [003] ..Zff   252.603811:
jiffies_access: (update_process_times+0x42/0xd0)
            tail-119     [003] ..Zff   252.603814:
jiffies_access: (update_process_times+0x52/0xd0)
            tail-119     [003] ..Zff   252.603818:
jiffies_access: (get_jiffies_update+0x30/0x120)
            tail-119     [003] ..Zff   252.603822:
jiffies_access: (rcu_sched_clock_irq+0x2d/0x660)
            tail-119     [003] d.Z..   252.603826:
jiffies_access: (calc_global_load_tick+0x10/0x60)
            tail-119     [003] d.Z..   252.603829:
jiffies_access: (sched_tick+0xec/0x2f0)
```

Wprobe Trigger: Dynamic `set_wprobe` Action (Experimental)

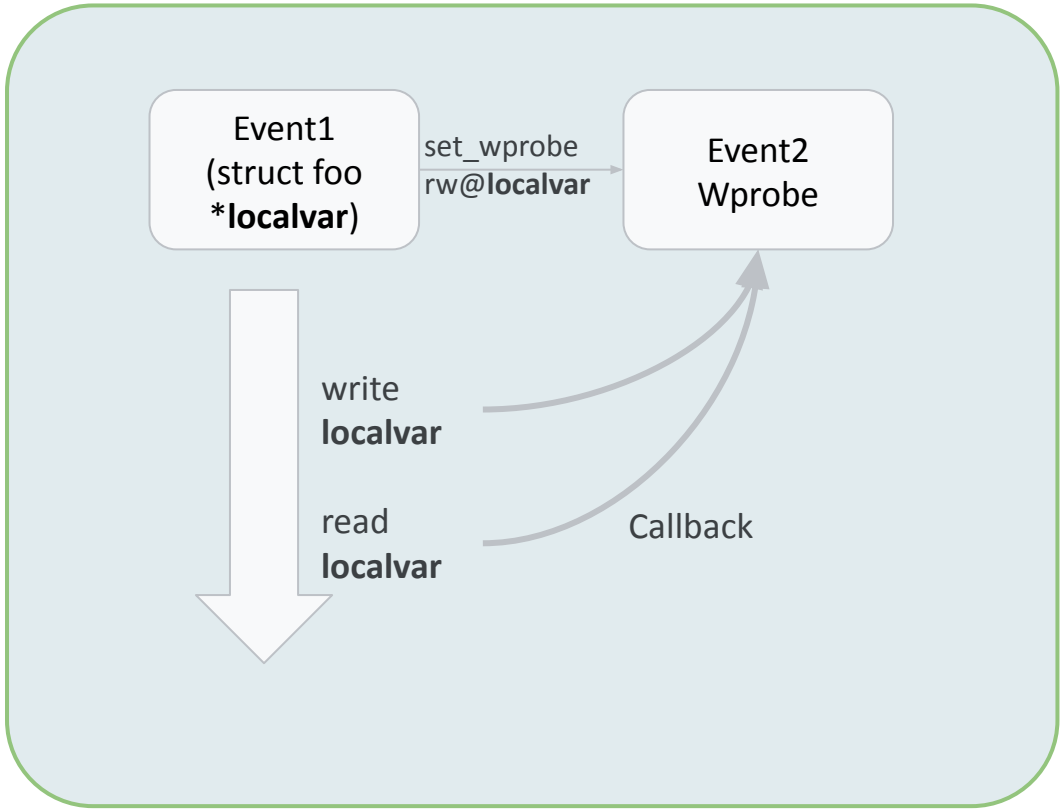
Dynamic Address Assignment

Trigger Action: The **set_wprobe** trigger action allows any other trace event (e.g., Kprobe or Fprobe on function entry) to dynamically specify the memory address for a Wprobe.

Runtime Configuration: The monitoring address is calculated at runtime by the triggering probe based on its current context (e.g., register value, stack frame offset).

The Debugging Breakthrough: This mechanism allows for the **real-time tracing of local function variables**.

Scenario: Probe `func_A` entry, calculate address of local variable `i` (on stack), and set Wprobe to monitor that exact address. Trace data changes without recompilation.



Synthesizing Advanced Tracing Benefits



Context-Aware Tracing

Features like **Kprobes+BTF** and **Eprobes** transform low-level register dumps into high-level, structured data, providing *what* is happening, not just *where* it is happening.



Minimal Performance Overhead

Optimizations like **Trapless Probes** and efficient **USDT/Uprobe** implementation ensure tracing can be "always-on" in production with negligible impact on system performance.



Dynamic Runtime Flexibility

Wprobe's `set_wprobe` action and the **Syscall Tracer's BTF support** enable dynamic instrumentation of memory and user-space contexts at runtime, facilitating deep, surgical debugging.

Dynamic Probe Feature Comparison (Summary)

Probe Type	Instrumentation Point	BTF Parameter Support	Primary Benefit / Status
Kprobes (Any)	Any instruction in the kernel	Entry/Exit	Most versatile, undergoing "Trapless" optimization.
Fprobes (Functions)	Function entry/exit only	Yes (Full)	Function entry/exit abstraction, consolidating logic.
Tprobes (Tracepoint)	Raw Tracepoints	Yes (Dereference)	Stable, static probe points with structured data access.
Uprobes (User-space)	Any instruction in user-space binaries	Yes (via BPF tools)	Deep user-space visibility, performance is the new focus.
Eprobes (Event)	Existing Trace Event	Yes (Pointer Access)	Probes existing trace events and accesses pointed-to data fields.
Wprobe (Watchpoint/Exp)	Memory Access (Load/Store)	Trigger (Monitors Address)	Allows monitoring of memory access, enabling event-triggered watchpoints via `set_wprobe`.

Conclusion: Impact & Outlook

The kernel tracing landscape is shifting from heavy, low-level breakpoints to highly optimized, BTF-aware instrumentation, enabling production-grade observability with minimal overhead and high-level, structured data.

Questions?

Thank you for your attention on this technical deep dive.